

Alphred 프로젝트 기능 명세서 및 시스템 아키텍처 기획서: **Antigravity** 개발 본부 전달용

본 문서는 Nous Research의 오픈소스 기반 자율형 AI 에이전트인 **Hermes Agent**를 핵심 추론 엔진으로 채택하여, 고도의 작업 스케줄링, 다중 디바이스 프론트엔드 통신, 그리고 외부 하위 서비스(Sub-service) 오케스트레이션 기능을 통합한 차세대 에이전트 프로젝트 '**Alphred**'의 상세 기능 명세 및 아키텍처 설계도이다. **Antigravity** 개발팀은 본 명세서를 바탕으로 단일 대화형 챗봇의 한계를 극복하고, 시스템 백그라운드에서 복잡한 연쇄 작업을 자율적으로 수행하며 우선순위를 조정하는 중앙 집중형 인공지능 운영 체제를 구축해야 한다.

Hermes Agent는 폐쇄형 학습 루프(Closed learning loop), 영구적 메모리 저장소, 경험 기반의 기술(Skill) 자체 생성, 그리고 70개 이상의 내장 도구를 지원하는 강력한 자율형 에이전트이다¹.

Alphred는 이러한 **Hermes**의 코어 생태계를 수정 없이 그대로 활용하되, 그 상단에 사용자 중심의 유연한 큐(Queue) 관리 미들웨어와 상태 제어 머신을 덧씌우는 래퍼(Wrapper) 패턴으로 설계된다. 이를 통해 인간의 지속적인 개입 없이도 시간과 중요도에 따라 작업을 배열하고, 안드로이드 앱 및 ESP32 임베디드 기기와의 원활한 통신을 보장하는 진정한 의미의 디지털 비서를 구현한다.

1. **Hermes** 기반 코어 엔진 통합 및 자동 업데이트 아키텍처

Alphred의 추론 및 도구 실행을 담당하는 두뇌는 **Hermes Agent**이다. **Hermes**는 전 세계의 오픈소스 기여자들에 의해 정기적으로 코드가 업데이트되며, 새로운 기능과 최적화가 지속적으로 반영되는 활발한 프로젝트이다⁴. **Alphred**가 항상 최첨단의 성능을 유지하면서도 시스템 안정성을 담보하기 위해서는, 코어 에이전트를 최신 버전으로 유지하는 자동화된 버전 관리 데몬(Daemon)이 시스템 백그라운드에 구축되어야 한다.

안정적인 자동 업데이트를 위해 **Alphred**는 **Hermes** 내부에 구현된 **hermes update** 파이프라인의 메커니즘을 심분 활용하고, 이를 시스템 무중단 아키텍처에 통합한다. **Hermes**의 업데이트 프로세스는 단순한 코드 다운로드를 넘어, 시스템의 무결성을 검증하고 실패 시 롤백을 수행하는 다단계 방어 기제를 갖추고 있다⁵.

업데이트 파이프라인 단계	상세 동작 메커니즘 및 안정성 보장 로직
사전 스냅샷 백업	업데이트 진행 전 ~/.hermes/ 디렉터리 내의 페어링 데이터, 메모리 파일(MEMORY.md), 커스텀 기술 파일(SKILL.md) 등 런타임 상태 파일의 경량 스냅샷을 .zip 아카이브로 자동 생성한다 ⁵ .
비간섭 로컬 변경사항 보존	updates.non_interactive_local_changes: stash 설정을 기본값으로 사용하여,

	사용자가 로컬에서 수정한 구성 파일이나 플러그인 코드를 임시 저장(Stash)한 뒤 최신 코드를 가져오고(Pull) 다시 병합하여 충돌을 방지한다 ⁵ .
구문 검증 및 자동 롤백	코드 가져오기가 완료된 후, Hermes 는 시스템 부팅에 필수적인 8개의 핵심 파일에 대해 구문 분석을 수행한다. 병합 충돌 마커나 파일 손상이 감지되면 즉시 git reset --hard 를 통해 업데이트 이전의 커밋 해시로 시스템을 안전하게 롤백한다 ⁵ .
의존성 재설치 및 SIGHUP 방어	uv pip install -e ".[all]" 명령을 통해 최신 파이썬 의존성을 재설치한다. 이 과정은 SIGHUP 신호를 무시하도록 설계되어, 네트워크 단절이나 터미널 종료 시에도 파이썬 환경이 반쯤 설치된 상태로 파손되는 것을 원천 차단한다 ⁵ .
무중단 게이트웨이 재시작	업데이트가 성공하면 systemd 또는 launchd 와 같은 서비스 관리자를 통해 게이트웨이를 자동으로 재시작한다 ⁵ .

Antigravity 개발팀은 이 업데이트 메커니즘을 호출할 때 발생할 수 있는 특수한 엣지 케이스를 반드시 고려해야 한다. 과거 **Hermes** 에이전트가 자체적으로 게이트웨이 재시작을 지시하는 크론 작업을 생성했을 때, **SIGTERM** 신호와 시스템의 자동 재시작(**KeepAlive**) 기능이 충돌하여 무한 재시작 루프에 빠지는 치명적인 버그(이슈 **#30719**)가 보고된 바 있다⁶. 이를 방지하기 위해 **Alphred**의 스케줄러 미들웨어는 에이전트가 직접 라이프사이클 제어 명령(**hermes gateway restart**, **systemctl restart** 등)을 예약하는 것을 페이로드 필터링을 통해 원천 차단해야 하며, 60초 내에 3회 이상의 비정상 **SIGTERM**이 감지될 경우 자동 재개(**Auto-resume**)를 비활성화하는 안전망을 구축해야 한다⁶.

2. 동적 작업 스케줄링 및 큐(**Queue**) 관리 시스템

Alphred 기획의 핵심이자 가장 높은 우선순위를 가지는 요구사항은 모든 작업에 대한 통제권을 사용자에게 부여하는 유연한 스케줄링 시스템의 구축이다. 기존 **Hermes**가 제공하는 크론(**Cron**) 스케줄링은 지정된 시간에 독립된 에이전트 세션을 생성하여 단발성 작업을 수행하는 데에는 매우 뛰어나지만⁷, 중간에 긴급한 작업이 유입되거나 진행 중인 작업을 중단해야 하는 복잡한 운영 체제 수준의 프로세스 제어에는 한계가 있다.

따라서 **Antigravity** 개발팀은 **Hermes**의 실행 계층(**Execution Layer**) 위에 독자적인 '우선순위 큐 관리자(**Priority Queue Manager**)'를 데이터베이스 기반으로 구축해야 한다.

2.1. 작업 상태 제어 모델 (**Task State Machine**)

모든 작업(단발성 사용자 명령, 크론 기반의 반복 작업, 하위 서비스 트리거)은 생성 즉시 고유한

UUID가 부여되며, 상태 제어 머신에 의해 엄격하게 관리된다. 사용자는 제공되는 프론트엔드 인터페이스를 통해 현재 큐의 대기열과 각 작업의 진행 상태를 투명하게 조회할 수 있다.

작업 상태 (State)	상태 정의 및 시스템 동작 원리	전환 가능 상태
Pending (대기 전)	큐에 등록되었으나 아직 추론 리소스(LLM) 및 샌드박스 터미널이 할당되지 않은 상태. 1~10단계의 우선순위(Priority) 값에 따라 큐 내에서 위치가 동적으로 재배열된다.	In-Progress, Discarded
In-Progress (진행 중)	에이전트가 작업을 할당받아 LLM 추론 및 도구(Tool) 호출을 수행 중인 상태. API를 통해 진행률(hermes.tool.progress) 및 실행 중인 도구의 이름이 프론트엔드로 실시간 스트리밍된다 ⁹ .	Paused, Completed, Discarded
Paused (일시 중지)	상위 우선순위 작업의 선점(Preemption) 프로세스 또는 사용자의 명시적 개입에 의해 작업이 보류된 상태. 현재까지의 세션 컨텍스트와 메모리가 직렬화되어 데이터베이스에 안전하게 보존된다.	In-Progress, Discarded
Completed (완료)	작업이 성공적으로 종료되고 최종 결과 텍스트 또는 데이터가 도출된 상태. 결과물은 메모리 시스템(MEMORY.md)에 병합되거나 지정된 엔드포인트(웹훅, 메시징 채널 등)로 전송된다 ⁷ .	(최종 상태)

Discarded (폐기)	사용자의 강제 취소 요청, 치명적인 시스템 오류, 또는 타임아웃 초과(<code>terminal.timeout</code> 도달 등)로 인해 작업이 큐에서 강제로 제거된 상태 ¹¹ .	(최종 상태)
-----------------------	---	---------

2.2. 선점형(Preemptive) 우선순위 제어 메커니즘

작업의 중요도는 1(최하/백그라운드 스크래핑 등)부터 10(최상/사용자의 즉각적인 대화 응답)까지의 레벨로 구분된다. 유연한 스케줄링의 핵심은 저순위 작업이 LLM 리소스와 샌드박스를 점유하고 있을 때, 고순위 작업이 유입되면 이를 안전하게 중단하고 자원을 회수하는 선점형 스케줄링(Preemptive Scheduling)을 구현하는 데 있다.

예를 들어, 에이전트가 레벨 3의 대규모 코드 리팩토링 작업을 In-Progress 상태로 진행 중일 때 사용자가 레벨 10의 '즉각적인 웹 검색 및 요약'을 요청할 경우, 시스템은 다음과 같이 동작해야 한다.

시스템은 진행 중인 레벨 3 작업의 현재 컨텍스트를 즉시 캡처한다. Hermes의 API 서버가 제공하는 `/v1/responses` 엔드포인트는 `previous_response_id` 파라미터를 통해 서버 측에 대화 상태(도구 호출 기록 및 결과 포함)를 영속적으로 보존하는 기능을 완벽하게 지원한다⁹. 스케줄러는 이 ID를 데이터베이스에 Paused 상태로 저장한 뒤, 에이전트의 현재 추론 스트림을 우아하게(Gracefully) 중단시킨다. 이후 레벨 10 작업에 리소스를 할당하고 상태를 In-Progress로 변경하여 Hermes 추론 엔진에 주입한다. 긴급 작업이 Completed 상태에 도달하여 리소스가 반환되면, 스케줄러는 큐를 재평가하여 Paused 상태였던 레벨 3 작업을 찾아내고, 저장해둔 `previous_response_id`를 활용하여 중단된 지점부터 컨텍스트 손실 없이 작업을 재개(In-Progress)한다.

이러한 상태 기반 큐 관리를 통해 사용자는 스마트폰 앱이나 웹 대시보드에서 드래그 앤 드롭으로 큐 순서를 시각적으로 조정할 수 있으며, 백그라운드 작업과 실시간 대화가 충돌 없이 공존하는 진정한 비동기 운영 체제 환경을 경험하게 된다.

3. 다중 프론트엔드 통합을 위한 API 서버화 설계

Alphred는 단일 노트북이나 PC에 종속되는 데스크톱 앱을 넘어, 사용자의 안드로이드 스마트폰, ESP32 기반의 탁상용 에이전트 호출 장치, 스마트 스피커, 홈 어시스턴트(Home Assistant) 등 다양한 물리적 엔드포인트와 통신할 수 있는 클라우드 기반의 통합 두뇌로 설계되어야 한다.

3.1. OpenAI 규격 호환 API 서버 (OpenAI-Compatible Endpoint)

다양한 디바이스와의 호환성을 확보하기 위해, Alphred는 Hermes에 내장된 API 서버 기능을 적극적으로 활성화하여 통신 계층을 구축한다. Hermes는 환경 변수

`API_SERVER_ENABLED=true` 및 `API_SERVER_KEY`를 설정하고 `hermes gateway` 커맨드로 실행할 경우, 8642번 포트를 통해 완벽하게 OpenAI API 규격(OpenAI-compatible HTTP endpoint)을 준수하는 서버를 가동한다⁹.

이는 프론트엔드 개발팀(Antigravity)에게 엄청난 이점을 제공한다. 안드로이드 앱이나 ESP32 펌웨어를 개발할 때 복잡한 커스텀 웹소켓 프로토콜을 바닥부터 작성할 필요 없이, 시중에 널리 공개된 검증된 OpenAI REST API 클라이언트 라이브러리(Java, Kotlin, C++)를 그대로 가져와

사용할 수 있기 때문이다. Base URL만 Alphred 서버의 주소(예: `http://<alphred-ip>:8642/v1`)로 변경하고 비밀키를 주입하면 즉각적인 통신이 성립된다¹².

지원 API 엔드포인트	호환 규격	프론트엔드 연동 주요 용도 및 데이터 처리 방식
POST /v1/chat/completions	OpenAI Chat Completions	단발성 명령의 전달 및 실시간 텍스트 스트리밍. 에이전트가 백그라운드에서 터미널을 실행하거나 웹 검색을 수행할 때, hermes.tool.progress 이벤트를 프론트엔드로 방출하여 UI에 '웹 검색 중...', '코드 실행 중...'과 같은 시각적 인디케이터를 렌더링할 수 있다 ⁹ .
POST /v1/responses	OpenAI Responses API	다중 턴(Multi-turn) 세션의 컨텍스트 유지. 클라이언트가 전체 대화 기록을 매번 전송할 필요 없이 previous_response_id 만 전달하면, 서버가 모든 도구 호출 결과와 이전 컨텍스트를 복원하여 추론을 이어나간다 ⁹ . 모바일 환경의 네트워크 대역폭 절약에 필수적이다.
GET /v1/runs/{run_id}	Custom Runs API	비동기 백그라운드 작업의 상태 폴링. 클라이언트가 스트리밍 연결을 유지하지 않고도 긴 작업의 성공 여부 및 최종 사용 토큰(usage) 정보를 조회할 수 있다 ⁹ .

3.2. 멀티모달 프론트엔드: 음성(Voice) 및 이미지 처리 파이프라인

ESP32 기반의 에이전트 호출 장치나 안드로이드 앱은 단순한 텍스트 챗봇을 넘어 음성 명령을 수신하고 음성으로 답변하는 하드웨어 인터페이스를 요구한다. **Hermes** 코어는 이미 고도화된

음성 모드(Voice Mode) 및 오디오 파이프라인을 내장하고 있으므로¹³, Alphred는 이를 API를 통해 프론트엔드에 노출해야 한다.

사용자가 안드로이드 앱이나 ESP32의 물리적 버튼을 눌러 발화하면, 기기는 녹음된 오디오 파일(예: .wav 또는 .ogg 포맷)을 멀티모달 API 규격에 맞추어 서버로 전송한다. Alphred 내부의 음성 인식(STT) 파이프라인은 설정에 따라 로컬 환경의 faster-whisper 모델을 사용하여 무료로 텍스트를 추출하거나, 속도가 중요한 경우 Groq Whisper API를 호출하여 즉각적인 전사를 수행한다¹⁴. 추출된 텍스트는 앞서 설명한 큐 관리자를 통해 처리되며, 최종 도출된 텍스트 답변은 다시 텍스트-음성 변환(TTS) 파이프라인으로 전달된다.

TTS 엔진의 경우, 오프라인 환경이나 비용 절감이 중요한 경우 Edge TTS 또는 로컬 기반의 NeuTTS(espeak-ng 백엔드)를 사용하여 음성 파일을 합성하고¹⁴, 고품질의 자연스러운 대화가 필요한 경우 ElevenLabs API를 활용할 수 있도록 유연하게 구성되어 있다¹⁶. 프론트엔드는 응답 페이로드에 포함된 음성 데이터를 스트리밍 받아 스피커로 출력함으로써 완벽한 양방향 음성 비서 경험을 완성한다.

더불어 카메라가 장착된 기기에서 시각적 분석을 요청할 경우, OpenAI의 멀티모달 규격인 {"type": "image_url", "image_url": "data:image/png;base64,..."} 형태의 페이로드를 서버로 전송하면, Alphred는 내부의 vision_analyze 도구를 활성화하여 이미지를 분석하고 결과를 반환한다⁹.

4. 하위 서비스(Sub-Service) 오케스트레이션 및 외부 프로그램 호환화

Alphred는 단순한 대화형 도구나 한정된 기술(Skill)의 실행을 넘어, 사용자가 직접 개발한 스크립트나 기업 내부의 레거시 시스템 등 외부에서 구동되는 독립된 프로그램을 능동적으로 제어하고 활용하는 오케스트레이션 능력을 갖추어야 한다. 이를 위해 Antigravity 팀은 Model Context Protocol (MCP)과 프로그래매틱 도구 호출(execute_code)이라는 두 가지 핵심 아키텍처를 결합하여 완벽한 하위 서비스 호환 계층을 설계해야 한다.

4.1. Model Context Protocol (MCP) 기반의 서비스 통합

MCP는 에이전트(클라이언트)가 외부의 독립적인 도구 서버와 구조화된 방식으로 통신할 수 있게 해주는 업계 표준 프로토콜이다¹⁸. 사용자가 파이썬이나 Node.js로 특정 기능을 수행하는 프로그램(예: 로컬 데이터베이스 분석기, IoT 기기 제어 데몬, 사내 인트라넷 크롤러 등)을 구축했다면, 해당 프로그램에 MCP 서버 인터페이스만 추가함으로써 Alphred가 이를 네이티브 도구처럼 인식하고 호출할 수 있다¹⁹.

Alphred의 ~/.hermes/config.yaml 내 mcp_servers 블록을 통해 하위 서비스의 연결 방식을 유연하게 정의할 수 있다. 하위 서비스가 Alphred와 동일한 머신 내에 위치한 실행 파일인 경우, stdio 전송 방식을 사용하여 하위 프로세스(Subprocess)를 직접 생성하고 표준 입출력 파이프를 통해 통신한다¹⁹. 반면, 하위 서비스가 원격 클라우드 서버나 별도의 도커(Docker) 컨테이너에 격리되어 있다면 HTTP(S) 기반의 전송 방식을 사용하며, 필요시 OAuth 2.1 인증 흐름이나 상호 TLS(mTLS) 클라이언트 인증서(client_cert)를 통해 안전한 통신 채널을 확보한다¹⁸.

에이전트가 구동되거나 설정이 갱신될 때, Alphred의 MCP 클라이언트는 등록된 하위 서비스들에 연결하여 사용 가능한 도구 목록(list_tools)을 자동 발견(Auto-discovery)한다¹⁹. 보안 및 예측 불가능한 오작동을 방지하기 위해, 시스템 관리자는 tools.include 및 tools.exclude

설정을 명시하여 하위 서비스가 노출하는 수많은 기능 중 검증되고 안전한 특정 기능(예: 읽기 전용 쿼리)에만 Alphred의 LLM 추론 엔진이 접근할 수 있도록 엄격한 방화벽을 구성해야 한다²⁰.

4.2. 프로그래매틱 도구 호출 (execute_code)을 통한 동적 워크플로우 제어

단순히 하위 서비스의 API를 한 번 호출하는 것을 넘어, 여러 하위 서비스의 결과물을 종합하여 복잡한 로직을 수행해야 할 때 Alphred는 execute_code 도구를 활용하여 극적인 효율성을 달성한다²². 이 기능은 에이전트가 직접 파이썬(Python) 스크립트를 동적으로 작성하고 이를 샌드박스 환경에서 실행하여 도구들을 프로그래밍 방식으로 조작할 수 있게 해준다²².

에이전트가 외부 하위 서비스 3곳에서 데이터를 가져와 비교 분석해야 한다고 가정해 보자. 일반적인 에이전트 루프에서는 LLM이 1번 서비스를 호출하고 응답을 읽은 뒤, 다시 추론을 거쳐 2번 서비스를 호출하는 방식의 비효율적인 다단계 추론(Multi-step reasoning)이 발생하며 막대한 토큰을 낭비한다. 반면 Alphred는 execute_code를 사용하여 단일 파이썬 스크립트 내에 3개의 하위 서비스를 순차적으로 호출하는 로직(For-loop, 조건 분기 등)을 작성한 뒤 이를 유닉스 도메인 소켓(Unix domain socket) RPC 통신을 통해 샌드박스에서 실행한다³.

스크립트는 300초의 엄격한 타임아웃과 50KB의 표준 출력(Stdout) 제한 하에 안전하게 실행되며, 외부 프로그램과의 연속적이고 복잡한 상호작용 과정에서 발생하는 대규모 중간 데이터는 스크립트 내부에서 폐기되고 오직 최종적으로 정제된 요약본만이 LLM의 컨텍스트 윈도우로 반환된다²². 또한 특정 하위 서비스 통신에 필요한 API 키나 인증 토큰은 보안을 위해 required_environment_variables 선언을 통해 안전하게 하위 프로세스로

패스스루(Passthrough)되어, 보안 태세를 약화시키지 않으면서도 강력한 연동을 지원한다²².

5. 특정 작업의 반복 스케줄링 및 주기적 큐(Queue) 등록

앞서 정의한 하위 서비스 호출 및 파이프라인 스크립트를 사용자가 매일, 매주, 특정 요일 등 정해진 주기에 자동으로 구동하고자 하는 요구사항은 Alphred의 주기적 스케줄러 시스템을 통해 구현된다. 이는 Hermes가 기존에 보유하고 있던 직관적인 크론(Cron) 스케줄링 엔진의 장점을 완벽히 흡수하면서도, 2장에서 설계한 우선순위 큐 관리자와 결합하여 더욱 고도화된 형태로 동작한다.

5.1. 크론 엔진과 작업 큐의 유기적 통합

Hermes 내부의 스케줄러는 사용자의 자연어 명령("every weekday morning at 9am")을 표준 크론 표현식(예: 0 9 ** 1-5)으로 변환하여 ~/.hermes/cron/jobs.json 파일에 원자적(Atomic)으로 저장하는 훌륭한 인터페이스를 갖추고 있다⁷.

Alphred 시스템은 게이트웨이의 백그라운드 스레드에서 60초마다 스케줄러 틱(Tick)을 발생시켜 완료된 작업을 확인한다⁸. 그러나 기본 Hermes처럼 틱 발생 즉시 에이전트를 강제로 실행시키는 대신, Alphred는 해당 작업을 지정된 우선순위(Priority) 값과 함께 상태 제어 머신의 'Pending(대기 전)' 큐에 편입시키는 방식으로 아키텍처를 우회(Intercept)해야 한다. 이를 통해 시스템 부하가 높거나 더 긴급한 실시간 사용자 대화가 진행 중일 경우, 백그라운드 반복 작업은 큐에서 유연하게 대기하며 시스템 자원의 병목을 방지한다.

5.2. 컨텍스트 격리 및 비간섭적 백그라운드 실행

큐에서 리소스를 할당받아 실행 상태(In-Progress)로 전환된 반복 작업은, 사용자의 현재 대화 기록(Conversation history)에 전혀 영향을 주지 않는 완전히 격리된 신규 에이전트 세션(Fresh

Agent Session) 내에서 실행된다⁸.

이러한 독립 세션에는 사전에 연결된 하위 서비스의 기술(Skill) 파일이 컨텍스트로 주입되어, 프롬프트에 구체적인 지시를 반복 기재하지 않아도 에이전트가 올바른 절차를 따라 작업을 수행하도록 유도한다⁷. 또한 반복 스케줄링으로 인한 무한 재귀 호출(Recursive scheduling explosion)을 방지하기 위해, 실행되는 격리 세션 내에서는 또 다른 스케줄을 생성하는 cronjob 도구 세트가 런타임에서 강제로 비활성화되는 안전 장치가 적용된다⁷.

주기적인 반복 작업의 결과는 사용자의 대화창을 차단(Block)하지 않고 비동기 하위 에이전트(Asynchronous Subagents) 형태로 처리된다²⁴. 작업이 무사히 완료되면 그 결과는 텔레그램, 디스코드 채널, 이메일 등 사전에 설정된 전달처(Delivery Target)로 조용히 발송되며⁷, 큐 상태는 성공적으로 Completed로 전환된다. 사용자는 원할 때 언제든지 프론트엔드 대시보드를 열어 과거에 주기적으로 실행된 작업들의 산출물(~/hermes/cron/output/ 경로 저장)을 열람하거나 감사(Audit)할 수 있다²⁶.

6. 결론 및 구현 방향성

프로젝트 Alphred는 단순한 질문-응답형 챗봇의 한계를 넘어, 복잡한 현대적 IT 인프라와 일상 생활의 요구사항을 동시에 충족시킬 수 있는 차세대 자율형 운영 체제의 청사진이다. Hermes Agent의 압도적인 추론 능력과 풍부한 도구 생태계를 코어 엔진으로 삼고, 그 위에 기업용 시스템 수준의 상태 기반 큐(Queue) 관리와 우선순위 선점 알고리즘을 도입함으로써, 동시다발적인 긴급 상황과 백그라운드 자동화 작업을 완벽히 조율해 낸다.

특히, OpenAI API 규격을 네이티브로 활용한 범용 엔드포인트 구축은 안드로이드 기기나 ESP32와 같은 다양한 엣지(Edge) 디바이스와의 연동 장벽을 허물어뜨리며, MCP 프로토콜과 프로그래매틱 도구 호출(execute_code)의 결합은 사용자가 보유한 어떠한 파편화된 외부 하위 서비스라도 에이전트의 손끝에서 오케스트레이션할 수 있는 강력한 자유를 부여한다.

Antigravity 개발 본부는 본 기획서를 명확한 이정표로 삼아, 인간의 명시적 개입을 최소화하면서도 일상의 모든 복잡한 작업을 능동적이고 유연하게 통제하는 초연결 디지털 비서를 성공적으로 구현해 내길 바란다.

참고 자료

1. A curated list of awesome skills, tools, integrations, and resources for Hermes Agent by Nous Research - GitHub, <https://github.com/OxNyk/awesome-hermes-agent>
2. Hermes WebUI: The best way to use Hermes Agent from the web or from your phone! - GitHub, <https://github.com/nosquena/hermes-webui>
3. The AI Agent That Learns While It Works — A Complete Guide to Hermes Agent, https://dev.to/aditya_007/the-ai-agent-that-learns-while-it-works-a-complete-guide-to-hermes-agent-n2n
4. Releases · NousResearch/hermes-agent - GitHub, <https://github.com/NousResearch/hermes-agent/releases>
5. Updating & Uninstalling | Hermes Agent - nous research, <https://hermes-agent.nousresearch.com/docs/getting-started/updating>
6. Agent can schedule gateway-restart cron job that kills its own runtime, creating respawn loop with launchctl/systemd KeepAlive · Issue #30719 - GitHub, <https://github.com/NousResearch/hermes-agent/issues/30719>

7. Scheduled Tasks (Cron) - Hermes Agent,
<https://hermes-agent.nousresearch.com/docs/user-guide/features/cron>
8. Cron Internals | Hermes Agent - nous research,
<https://hermes-agent.nousresearch.com/docs/developer-guide/cron-internals>
9. API Server | Hermes Agent - nous research,
<https://hermes-agent.nousresearch.com/docs/user-guide/features/api-server>
10. Migrate from OpenClaw | Hermes Agent - nous research,
<https://hermes-agent.nousresearch.com/docs/guides/migrate-from-openclaw>
11. Configuration | Hermes Agent - nous research,
<https://hermes-agent.nousresearch.com/docs/user-guide/configuration>
12. Hermes Agent - Open WebUI,
<https://docs.openwebui.com/getting-started/quick-start/connect-an-agent/hermes-agent/>
13. Voice Mode | Hermes Agent - nous research,
<https://hermes-agent.nousresearch.com/docs/user-guide/features/voice-mode>
14. Use Voice Mode with Hermes,
<https://hermes-agent.nousresearch.com/docs/guides/use-voice-mode-with-hermes>
15. Hermes Agent — Configure Voice Control from Your Phone | by Rost Glukhov - Medium,
<https://medium.com/@rosgluk/hermes-agent-configure-voice-control-from-your-phone-87d6a71d2ed8>
16. Voice & TTS | Hermes Agent - nous research,
<https://hermes-agent.nousresearch.com/docs/user-guide/features/tts>
17. Tools & Toolsets | Hermes Agent - nous research,
<https://hermes-agent.nousresearch.com/docs/user-guide/features/tools>
18. MCP (Model Context Protocol) | Hermes Agent - nous research,
<https://hermes-agent.nousresearch.com/docs/user-guide/features/mcp>
19. native-mcp | Skills Marketplace - LobeHub,
<https://lobehub.com/skills/nousresearch-hermes-agent-native-mcp>
20. MCP Config Reference | Hermes Agent - nous research,
<https://hermes-agent.nousresearch.com/docs/reference/mcp-config-reference>
21. Use MCP with Hermes,
<https://hermes-agent.nousresearch.com/docs/guides/use-mcp-with-hermes>
22. code-execution.md - NousResearch/hermes-agent - GitHub,
<https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/features/code-execution.md>
23. How to Build a Cron-Based AI Automation with Hermes Agent: Scheduling and Skills,
<https://www.mindstudio.ai/blog/build-cron-based-ai-automation-hermes-agent>
24. Hermes Agent Adds Asynchronous Subagents, So Delegated Work No Longer Blocks the Parent Chat - MarkTechPost,
<https://www.marktechpost.com/2026/06/16/hermes-agent-adds-asynchronous-subagents-so-delegated-work-no-longer-blocks-the-parent-chat/>
25. Features Overview | Hermes Agent,

<https://hermes-agent.nousresearch.com/docs/user-guide/features/overview>

26. Cron scheduling - Hermes Agent,

<https://nousresearch-hermes-agent.mintlify.app/user-guide/features/cron>